

Lab 3: Component Modelling & Architectural Pattern Selection

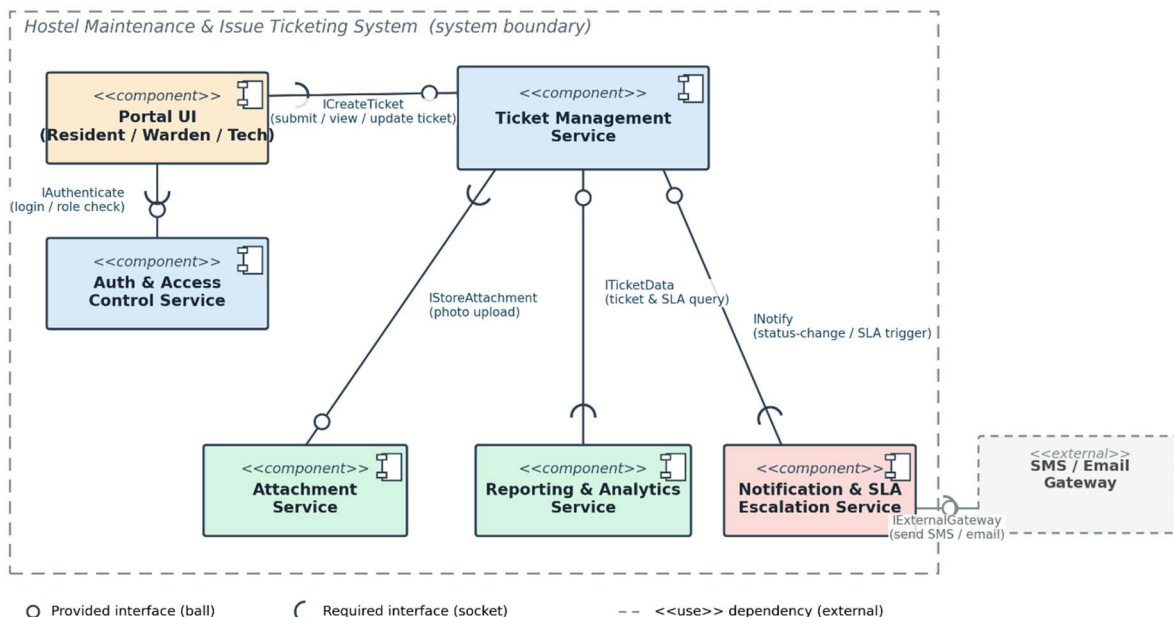
Problem Statement #07: Hostel Maintenance & Issue Ticketing System

1. Component Diagram

Unlike the in-class kiosk example, this problem statement doesn't hand us any components to start with, so we went back to our own Lab 1/Lab 2 requirements table (FR-001 to FR-005, NFR-001, NFR-002) and picked out one component for each major responsibility. We ended up with 6 components:

- Portal UI (Resident / Warden / Technician) — the shared web/mobile front-end all three roles log into, just with different dashboards (this maps to our role-based dashboard routing from FR-002)
- Auth & Access Control Service — institution-login + role check, keeps this logic in one place instead of repeating it everywhere (FR-002)
- Ticket Management Service — the core: create ticket, update status, assign technician, track lifecycle (FR-001, FR-003) — basically the hub everything else talks to
- Attachment Service — handles the photo upload residents attach to a ticket (FR-001)
- Notification & SLA Escalation Service — sends the status-change alerts and checks for SLA breaches past 24 hours (FR-004, NFR-001)
- Reporting & Analytics Service — generates the Warden's weekly/monthly reports and SLA compliance numbers (FR-005)

The diagram below shows all 6 components inside the system boundary, plus the external SMS/Email Gateway that the Notification service depends on for actually sending the alert. Ball notation = provided interface, socket notation = required interface, and every connector is labelled with what it's actually sending across — that's 6 interfaces shown in total, above the minimum of 4.



2. Written Justification

Architecture Selection: *“We chose Microservices Architecture for the Hostel Maintenance & Issue Ticketing System.”*

Architectural Choice

We went with Microservices this time, which is different from the layered approach we used for the coffee kiosk in the earlier exercise — mainly because this isn't a single device doing one job, it's an institution-wide system with three different user roles, a background SLA checker, and reporting that runs on its own schedule. Each of our 6 components is its own independently deployable service that talks to the others only through its interface.

Reason 1 – the workloads are genuinely different from each other

Ticket creation happens constantly (every resident complaint), SLA escalation has to run continuously in the background checking timers, and reporting only runs weekly/monthly when the Warden asks for it. These don't share a load pattern at all, so bundling them into one layered app means the whole app has to scale for the busiest one. As separate services we can scale Ticket Management during peak hostel-issue hours (like after a power cut) without touching Reporting.

Reason 2 – fault isolation actually matters here

If the Reporting & Analytics service crashes while generating a big monthly PDF, that shouldn't stop a resident from logging a burst pipe as an emergency ticket. In a tightly layered/monolithic setup a bug in one layer can bring the whole app down; with microservices, Ticket Management and Notification can keep running independently even if Reporting is having a bad day.

Security Advantage

Because Auth & Access Control is pulled out as its own service, every other service (Ticket Management, Reporting, etc.) has to call `IAuthenticate` rather than implement its own login/role check. That means there's exactly one place enforcing “Residents can't see Warden controls,” which was literally our FR-002 acceptance criteria — instead of that check being copy-pasted (and possibly forgotten) in multiple services.

Performance Benefit

Since Notification & SLA Escalation is a separate service, it can keep polling for 24-hour SLA breaches in the background without slowing down the actual ticket-submission API that residents are waiting on. If that logic lived inside the same process as Ticket Management, every scan for overdue tickets would be competing for the same resources as someone just trying to submit a ticket — splitting them keeps the resident-facing path fast, which matters more for something urgent like plumbing/electrical issues.